RAPPORT PROJET IoT
- Serre Intelligente -

Réseau local 802.15.4 (Thread) : QoS
et télémétrie vers le cloud

Objectif

Le présent projet vise à concevoir et valider une architecture IoT complète reposant sur un réseau local basse consommation 802.15.4/Thread. L'architecture doit permettre la collecte de télémétrie et la remontée d'événements critiques, avec gestion de priorités (QoS) au niveau de la passerelle, puis export des données vers le cloud via un uplink NB-IoT ou LoRaWAN.

Nous avons choisi de simuler une serre intelligente avec plusieurs capteurs qui vont nous permettre de faire remonter les données vitales des plantes en temps réel et pouvoir les traiter dans un second temps au niveau supérieur.

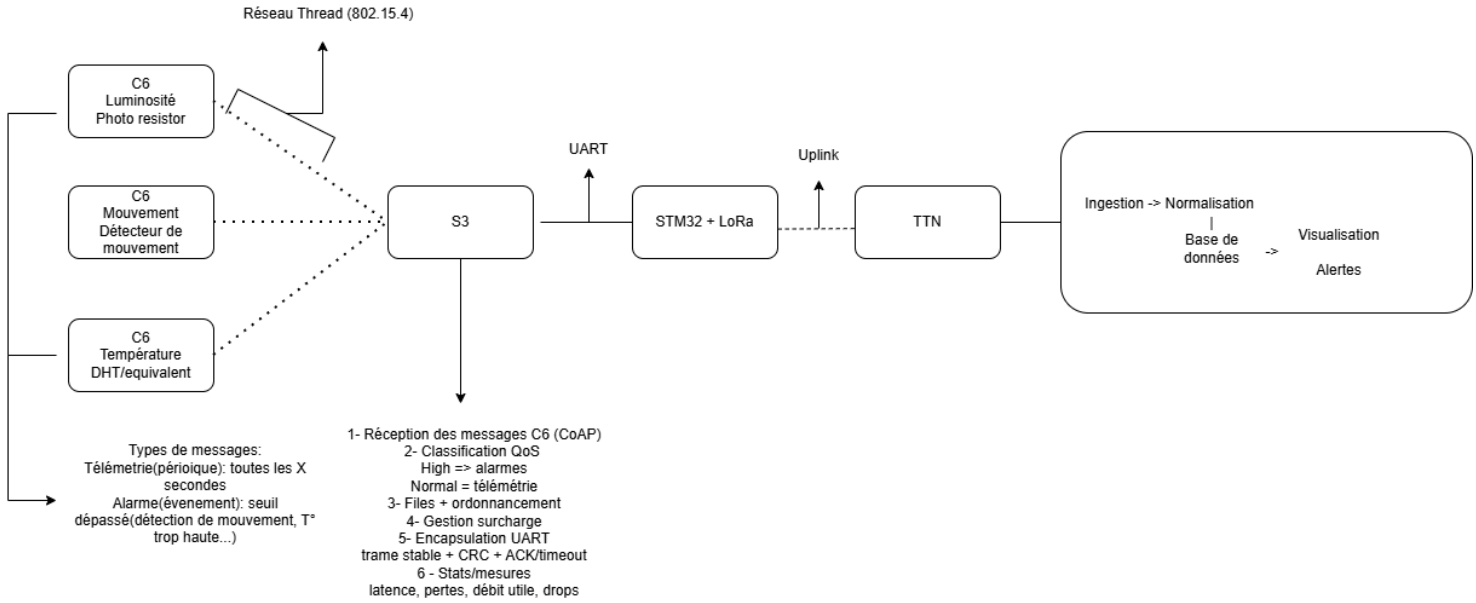
Finalité

Dans ce déploiement IoT réel, plusieurs capteurs partagent un même médium radio et produisent des flux hétérogènes :

- télémétrie périodique (mesures régulières),
- alarmes/événements (flux critiques, non réguliers).

Ce projet a donc pour finalité de démontrer qu'en présence de contraintes (débit, congestion), le système :

- préserve les messages critiques,
- dégrade proprement la télémétrie (compactage, rejet contrôlé),
- fournit une télémétrie exploitable côté cloud (structure, métadonnées, traçabilité).



Télémetrie

Le travail réalisé s'inscrit dans une logique de collecte de mesures issues de capteurs hétérogènes (capteur de température/humidité, capteur de luminosité et capteur de mouvement) et de remontée de ces données vers une passerelle.

Dès le départ, une contrainte matérielle a structuré l'architecture : une partie des capteurs disponibles fonctionne en 5V, alors que les microcontrôleurs ESP32 (C6/S3) opèrent en 3.3V sur leurs entrées/sorties. Cette incompatibilité de niveaux logiques rend risqué un câblage direct capteur-ESP32 sans adaptation.

Le choix a donc été fait de séparer la chaîne en deux domaines : acquisition capteurs 5V d'un côté, et communication radio / agrégation de l'autre.

À partir de cette contrainte, l'architecture retenue est la suivante :

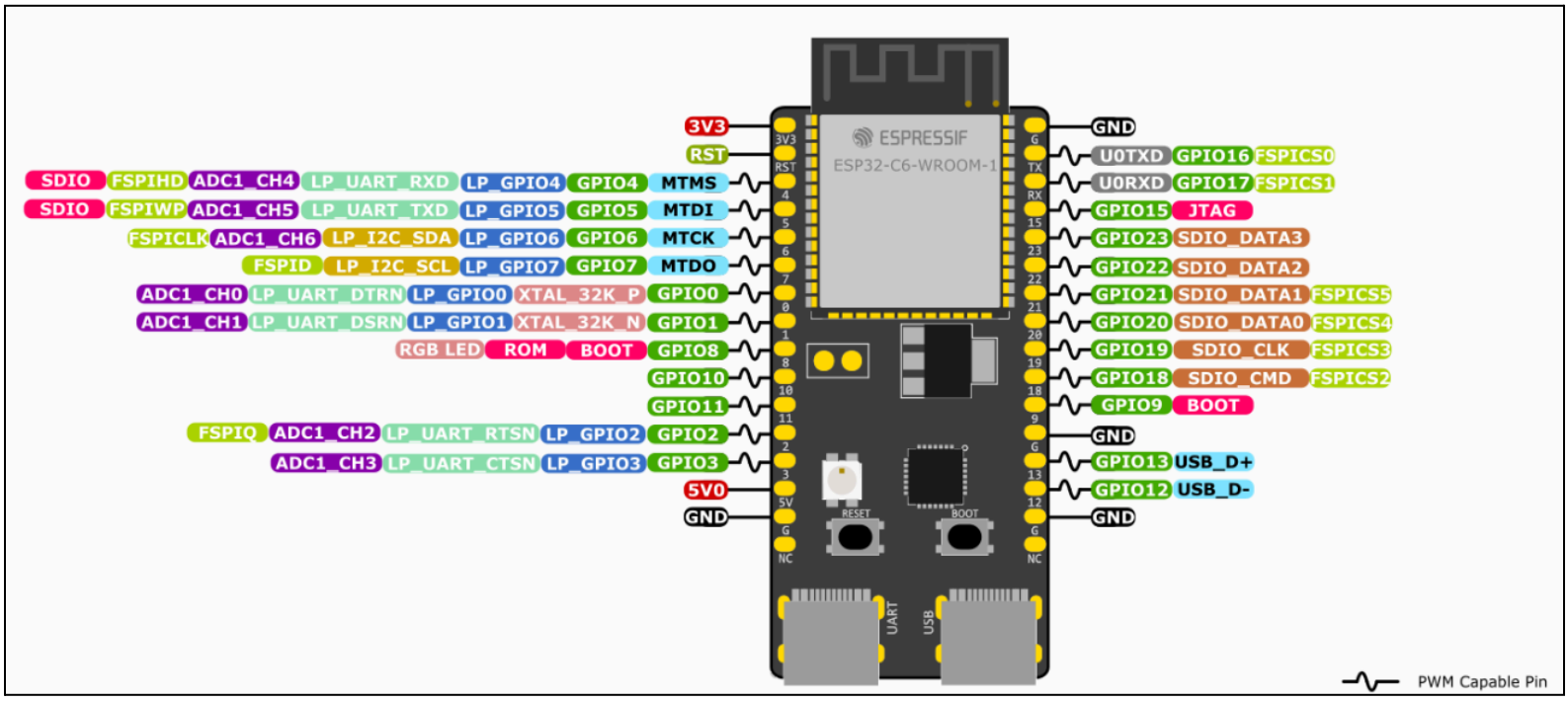
capteur 5V → Arduino Mega (alimentation + acquisition) → UART → ESP32-C6 (nœud capteur) → BLE → ESP32-S3 (passerelle).

L'Arduino joue le rôle d'interface d'acquisition, en fournissant l'alimentation 5V et en réalisant les lectures de capteurs. Les ESP32-C6, eux, sont dédiés à la publication des mesures vers la passerelle en radio. Cette séparation a été préférée à un montage "tout sur ESP32" car elle réduit les risques électriques et accélère la mise en route des capteurs.

Une fois la répartition des rôles établie, il fallait choisir un lien fiable entre l'Arduino et chaque ESP32-C6. L'UART a été retenu parce qu'il s'agit d'un transport point-à-point simple, déterministe et très facile à diagnostiquer. Concrètement, l'Arduino envoie une trame texte sur une ligne série et le C6 la réceptionne puis la traite. Ce choix a aussi l'avantage de découpler complètement l'acquisition et la publication radio : les capteurs peuvent être lus à une cadence fixe côté Arduino, tandis que l'ESP32-C6 publie lorsque la donnée est valide et disponible.

→ Une difficulté importante est apparue lors des premiers essais : les broches TX/RX affichées sur certaines cartes ESP32 correspondent souvent à l'UART de console déjà connecté au port USB (flash + logs). Câbler l'Arduino sur ce même UART provoque des conflits (données mélangées, upload instable, monitoring impossible).

Pour résoudre cela, l'UART de l'Arduino a été basculé sur UART1 côté ESP32-C6, avec des broches dédiées GPIO4 (RX) et GPIO5 (TX), de manière à conserver l'USB comme canal de debug indépendant.



Après le choix du transport UART, la question du format de données s'est posée. Un format ASCII KEY=VALUE séparé par ; a été adopté, avec un framing basé sur la fin de ligne (\n \r).

Exemple :
T=22.5;H=55, LUM=287;ETAT=SOMBRE, MOTION=1

Ce choix est volontairement pragmatique : la lisibilité permet de vérifier rapidement les valeurs et de diagnostiquer des erreurs de câblage ou de capteur sans outil spécialisé. De plus, le parsing côté ESP32-C6 reste simple (recherche de sous-chaînes T=, H=, etc., puis conversion).

Ce point a également révélé une limite acceptée à ce stade : l'absence de longueur, de CRC ou de numéro de séquence signifie que la robustesse n'est pas garantie en cas de corruption de trame ou de désynchronisation. Pour compenser partiellement, un mécanisme d'acquittement minimal a été introduit : après parsing, le C6 renvoie ACK\n sur l'UART. Cette décision prépare l'évolution vers un protocole fiable (ACK/timeout/retransmission), même si, sans SEQ et sans logique de retry côté Arduino, il s'agit encore d'une validation fonctionnelle plus que d'une QoS UART complète.

Lien ESP32-C6 vers ESP32-S3 : BLE

Le BLE a été retenu comme technologie de communication entre les nœuds capteurs (ESP32-C6) et la passerelle (ESP32-S3) pour trois raisons principales. D'abord, c'est la solution la plus simple à mettre en œuvre rapidement sur ESP32 : elle ne nécessite ni infrastructure (pas de routeur, pas de configuration IP), ni mise en place d'un réseau complexe, tout en permettant d'obtenir un lien radio opérationnel et testable immédiatement. Ensuite, BLE est très compatible avec le fonctionnement recherché dans cette phase : plusieurs nœuds "capteurs" en périphériques qui publient des données vers une passerelle centralisée qui collecte et agrège. Enfin, BLE est un protocole sécurisable proprement : il supporte les mécanismes standards (pairing/bonding, chiffrement des liens, gestion des clés) et peut donc être durci lorsque l'architecture fonctionnelle est validée.

Dans cette architecture, chaque ESP32-C6 joue le rôle de périphérique BLE (GATT Server) et l'ESP32-S3 celui de central (GATT Client). Concrètement, chaque C6 annonce sa présence en advertising sous un nom (ex. GreenHouse-C6-1) et expose un service BLE dédié.
La passerelle S3 scanne l'environnement, détecte les nœuds disponibles, se connecte à eux, puis récupère les données. Ce modèle correspond exactement à l'usage typique du BLE : des périphériques embarqués publiant une donnée, et un central responsable de l'agrégation.

Fonctionnement BLE :

Le code du C6 fonctionne parce qu'il est structuré autour de trois mécanismes simples qui se complètent.

1) Réception UART et reconstruction de trames :

Sur le C6 on configure l'UART avec : RX=GPIO4, TX=GPIO5, BAUD=9600. La tâche UART lit des octets en boucle, reconstruit une ligne jusqu'à \n , puis parse un format texte. Par exemple, si l'Arduino envoie :

```
T=22.5;H=55;LUM=287;ETAT=SOMBRE
```

alors le C6 extrait les champs en recherchant les tags (T=, H=, LUM=, ETAT=) et met à jour ses variables globales (g_temp, g_hum, g_lum, g_etat). Cette approche tient parce qu'elle repose sur un framing clair (fin de ligne) et un parsing robuste à l'ordre des champs.

2) Publication BLE via un serveur GATT :

Le C6 expose un service et une caractéristique qui représente “la dernière donnée capteur”. Deux comportements sont fournis :

READ : si la passerelle lit la caractéristique, on_read() renvoie la string formatée.
NOTIFY : dès qu’une nouvelle mesure arrive (après parsing UART), envoyer_ble() pousse la string au central si celui-ci s’est abonné.

Par exemple, la trame peut contenir :

T=22.5;H=55 ou LUM=287;ETAT=SOMBRE ou MOTION=1

Ce fonctionnement est stable parce que la stack NimBLE prend en charge toute la partie protocolaire (GAP/GATT, subscriptions, encapsulation ATT). Le code applicatif ne fait que définir les UUID, fournir le callback de lecture, et envoyer une notification quand nécessaire.

3) Logs et acquittement côté UART :

Après parsing, un log (ESP_LOGI) affiche les valeurs reçues pour validation. Ensuite, le C6 renvoie ACK\n sur l’UART. Cet ACK confirme que la trame a été reçue et traitée, mais il reste symbolique tant qu’on n’a pas ajouté côté Arduino un mécanisme complet (timeout, retransmission, et idéalement numéro de séquence).

→ **Modèle mental BLE : GAP pour se connecter, GATT pour transporter la donnée**

Pour comprendre le BLE proprement, il faut distinguer deux niveaux.

GAP (connexion) : Le C6 diffuse en advertising “je suis GreenHouse-C6-1 et j’expose un service X”. La passerelle S3 scanne, voit l’annonce, puis se connecte. Les callbacks (ex. gap_event_cb) gèrent connexion, déconnexion et abonnement aux notifications.	GATT (données) : Une fois connecté, la donnée est accédée via une arborescence : Service (ex. 0x00FF) Characteristic (ex. 0xFF01) avec READ et NOTIFY

Brièvement :
Le S3 lit la characteristic → reçoit T=22.5;H=55 (lecture “snapshot”).
Le S3 s’abonne → reçoit automatiquement à chaque nouvelle mesure des notifications :
T=22.6;H=55 puis T=22.6;H=56…

Que se passe-t-il si on a plusieurs ESP32-C6 ?

Avec plusieurs C6 (un par capteur), le S3 (central) scanne et voit plusieurs périphériques annonçant le même service. Il peut se connecter à chacun, puis s’abonner aux notifications. En pratique :

On remarque que le fonctionnement reste stable mais plus le nombre de connexions augmente, plus le coût mémoire/CPU côté central augmente (limite de connexions simultanées). Pour notre cas, cela ne pose pas problème.

Une conséquence immédiate est l’identification : si tous les nœuds annoncent le même nom et les mêmes UUID, cela fonctionne, mais la passerelle doit distinguer les sources soit par l’adresse MAC BLE ou un champ applicatif dans la payload.

Le choix d’un champ applicatif ID a été fait et notre trame a pris la forme suivante :

ID=1;T=22.5;H=55 → Trame envoyée par le capteur 1 de température et humidité ID=2;LUM=287;ETAT=SOMBRE→ Trame envoyée par le capteur 2 de luminosité ID=3;MOTION=1→ Trame envoyée par le capteur 3 de mouvements (intrusion)

Ce champ ID simplifie fortement l'agrégation et prépare les étapes suivantes (QoS, statistiques, export cloud).

Un point important est que NOTIFY n'est pas une livraison garantie : c'est best effort. Dans un contexte chargé (beaucoup de nœuds, radio bruitée), on peut observer : des retards de notification, des pertes si le central est débordé, des reconnexions.

Si on veut améliorer la fiabilité, deux approches existent :

- utiliser des Indications (confirmées au niveau BLE),
- OU ajouter un SEQ= applicatif pour détecter les pertes et reconstruire des statistiques.

Exemple :

```
ID=2;SEQ=1042;LUM=287;ETAT=SOMBRE
```

→ Nous avions opté pour cette trame dans notre projet

La carte ESP32-S3

Si les nœuds C6 sont les “capteurs distants”, l’ESP32-S3 agit comme le “cerveau” ou la passerelle (Gateway) du système. Son rôle est double : agréger les données provenant du réseau local BLE et assurer leur transmission sécurisée vers l’étage suivant (STM32/LoRaWAN).

1) Rôle de Central BLE et Agrégation

Contrairement aux C6 qui sont des périphériques esclaves, l’S3 fonctionne en mode Central (GATT Client). Il scanne activement l’environnement pour identifier les annonces `GreenHouse-C6-X`. Une fois la connexion établie, il réalise automatiquement la séquence suivante : négociation MTU (configurée à 256 octets), découverte du service GATT `0x00FF`, découverte de la caractéristique `0xFF01`, puis activation des notifications via l’écriture du CCCD. En cas de déconnexion, le scan est rellancé immédiatement.

L’agrégation repose sur un tableau de structures `node_data_t` (3 entrées, une par capteur). Le champ `ID=` de chaque trame déchiffrée permet d’identifier la source et de mettre à jour l’entrée correspondante : `ID=1` (température/humidité), `ID=2` (luminosité), `ID=3` (mouvement). Cela permet de disposer, à tout instant, d’une image complète de l’état de la serre sur un seul processeur.

2) Déchiffrement AES-128-CBC

Conformément aux exigences de confidentialité du projet, les payloads reçues en BLE sont chiffrées côté C6. La fonction `decrypt_and_process()` embarque une bibliothèque cryptographique (`mbedtls`) pour traiter les messages reçus. Les 16 premiers octets de la payload constituent le vecteur d’initialisation (IV), les octets suivants constituent le ciphertext, déchiffré avec la PSK (Pre-Shared Key de 128 bits) partagée entre tous les nœuds. Le padding PKCS7 est ensuite retiré pour obtenir le texte clair. Si la valeur de padding est invalide ou si `mbedtls` retourne une erreur, la trame est rejetée et un log `ESP_LOGE` est généré, constituant la journalisation des événements de sécurité.

3) Priorisation et trame de sortie vers le STM32

L’S3 ne se contente pas de transmettre ; il analyse la donnée pour optimiser la remontée vers le STM32. Une donnée critique comme une détection de mouvement (`MOTION=1`) est immédiatement marquée avec une priorité haute (`PRIO=H` pour High), tandis qu’une mesure de télémétrie classique est marquée en priorité normale (`PRIO=N` pour Normal). La trame de sortie envoyée au STM32 via UART prend la forme suivante :

```
S1;SEQ=<n>;PRIO=<H|N>;TYPE=TEL;T=<val>;H=<val>;LUM=<val>;MOT=<0|1>\n
```

Le champ `SEQ` est un compteur monotone incrémenté à chaque envoi confirmé. Cela permet au STM32 de détecter d'éventuelles pertes de paquets entre la passerelle et le module radio.

4) Liaison UART vers le STM32 et mécanisme d'acquittement

La communication vers le STM32 se fait sur `UART_NUM_1` avec les broches GPIO43 (TX) et GPIO44 (RX) à 115200 bauds. Après chaque envoi, l'S3 attend une réponse de la forme `ACK;SEQ=<n>` pendant 2 secondes (timeout). Si l'ACK est validé, le compteur de séquence est incrémenté. Sinon, un log d'erreur est généré et la trame n'est pas recomptée, permettant une retransmission à la prochaine notification BLE.

5) Tâche de monitoring

Pour faciliter la maintenance, l'S3 intègre une tâche FreeRTOS `monitor_task` qui tourne en arrière-plan. Toutes les 10 secondes, elle affiche sur la console série un résumé de l'état de chaque nœud : dernières valeurs reçues, ou `---` si aucune donnée n'a encore été reçue (nœud non connecté ou perte BLE). Cela permet de diagnostiquer rapidement une absence de données sans avoir à analyser les logs en continu.

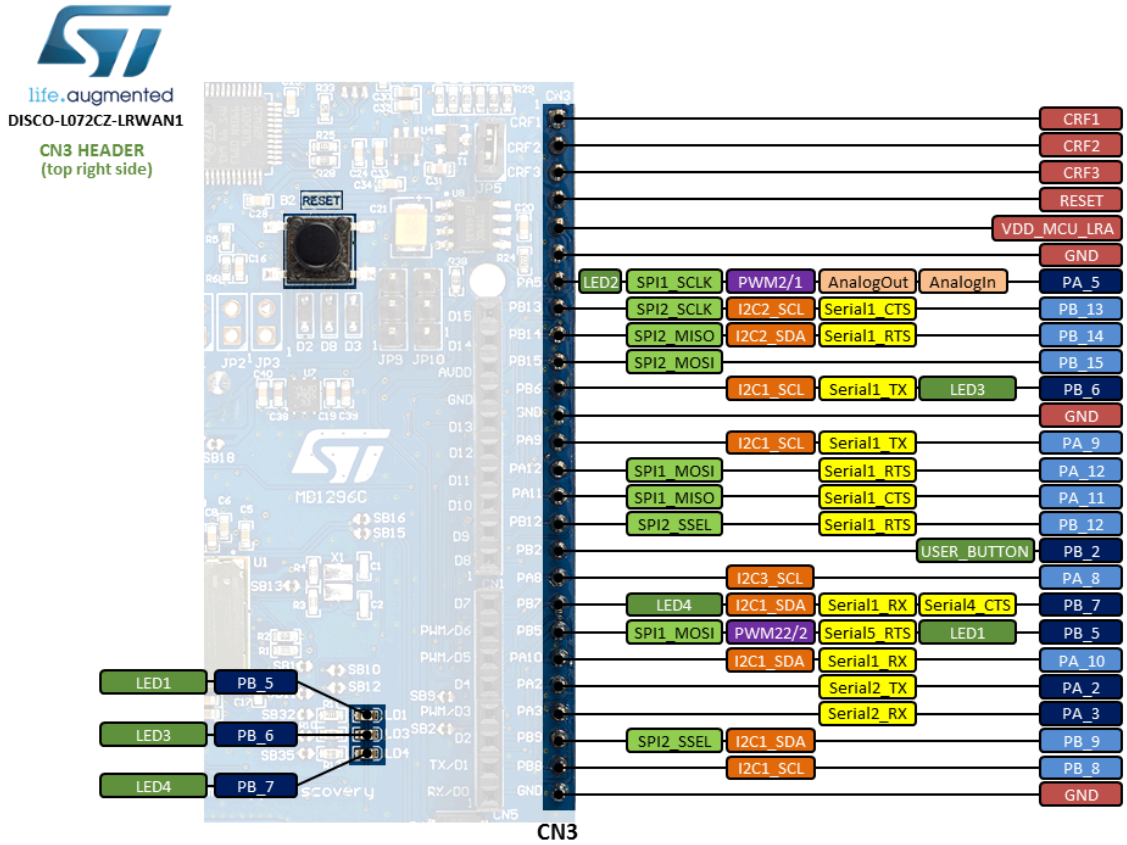
Réception des données & envoi via LoraWAN

Passerelle LoRaWAN — STM32 B-L072Z-LRWAN1

La carte STM32 B-L072Z-LRWAN1 constitue le maillon de transmission radio du système. Son rôle est d'assurer le pont entre la carte ESP32-S3, qui collecte et traite les données, et l'infrastructure LoRaWAN de The Things Network (TTN), qui les achemine vers l'application finale. Elle ne réalise aucun traitement applicatif sur les données : elle les transporte de manière fiable en respectant les contraintes du protocole LoRaWAN.

Réception des données depuis l'ESP32

La communication entre les deux cartes repose sur une liaison série UART cadencée à 115200 bauds. Le STM32 écoute en permanence sur les broches PA_9 (RX) et PA_10 (TX), et sonde l'UART toutes les 200 millisecondes via une EventQueue Mbed OS. Les trames sont délimitées par un caractère de saut de ligne (`\n`) et peuvent atteindre jusqu'à 51 octets, ce qui correspond à la limite maximale du payload LoRaWAN en SF7 sur la bande EU868. Toute trame dépassant cette taille est ignorée afin d'éviter les erreurs de transmission.



Connexion au réseau LoRaWAN

La carte rejoint le réseau TTN via la procédure OTAA (Over The Air Activation). Au démarrage, elle émet une Join Request contenant son DevEUI et son AppEUI. TTN vérifie l'identité du device à l'aide de l'AppKey, puis répond par un Join Accept qui établit une session chiffrée. Les clés de session sont ensuite dérivées automatiquement par les deux parties. Cette procédure garantit que seuls les devices enregistrés sur TTN peuvent accéder au réseau, et que les communications sont protégées contre l'écoute et la falsification.

L'Adaptive Data Rate (ADR) est activé, ce qui permet à TTN d'ajuster dynamiquement le datarate en fonction de la qualité du signal reçu. En environnement proche d'une gateway, le réseau utilisera un datarate élevé (SF7) pour minimiser le temps d'antenne. En cas de signal dégradé, il basculera vers un datarate plus robuste (SF12) au prix d'une trame plus longue.

Envoi des trames

Dès qu'une trame est reçue de l'ESP32, elle est transmise au réseau via un uplink non confirmé (unconfirmed uplink). Ce mode a été retenu pour limiter la consommation du duty-cycle : un uplink confirmé nécessiterait en effet une fenêtre de réception supplémentaire pour acquitter chaque message. En cas de blocage temporaire dû au duty-cycle EU868 — qui impose légalement une limite d'émission de 1% sur la bande 868 MHz — la stack Mbed OS détecte automatiquement la situation et la carte réessaie l'envoi après un délai de 3 secondes.

Architecture logicielle

Le firmware est développé sous Mbed OS et compilé via Keil Studio Cloud. L'ensemble des événements LoRaWAN (connexion, envoi, réception, erreurs) est traité de manière asynchrone au sein d'une EventQueue, ce qui évite tout blocage du fil d'exécution principal et garantit la réactivité de la lecture UART. Un port série USB (ST-Link, 115200 baud) permet de suivre en temps réel l'état de la carte pendant le développement et le débogage.

Partie Cloud

Vue d'ensemble

La partie cloud du projet **IoT Greenhouse** correspond à l'infrastructure serveur déployée sur un **VPS distant**, accessible via le domaine **vps.rayanemerlin.com**. Elle permet de recevoir, stocker et visualiser les données issues des capteurs LoRaWAN.

L'architecture repose sur **six conteneurs Docker** orchestrés avec **Docker Compose**. Les services communiquent via un réseau interne et sont exposés à l'extérieur uniquement par **Nginx**, qui sert de point d'entrée HTTPS.

Les principaux composants sont :

- **Nginx** : reverse proxy et gestion HTTPS
- **API Django** : réception des données TTN
- **RabbitMQ** : file de messages
- **Worker** : traitement des messages
- **InfluxDB** : stockage des données temporelles
- **Grafana** : visualisation des données

Le flux de traitement est le suivant : TTN envoie un webhook à l'API, l'API publie les données dans RabbitMQ, un worker les consomme et les écrit dans InfluxDB, puis Grafana interroge la base pour afficher les graphiques.

Cette architecture permet de séparer les responsabilités et d'améliorer la **résilience et la performance** du système.

Ingestion des données

L'API est développée avec **Django** et sert principalement à recevoir les **webhooks envoyés par The Things Network (TTN)** via l'endpoint :

POST /api/webhook/

Le message contient l'identifiant du capteur (`dev_eui`), le timestamp, les données capteurs (température, humidité, niveau d'eau) ainsi que des informations radio.

Après réception, l'API ne stocke pas directement les données. Elle les publie dans une **file RabbitMQ** afin de découpler la réception du traitement. Cela permet de répondre rapidement au webhook et d'éviter les pertes de données en cas de surcharge.

Un **worker dédié** consomme ensuite les messages de la file et les écrit dans **InfluxDB**. Le système utilise des mécanismes d'acquittement afin de garantir que les messages sont correctement traités.

Stockage des données

Les données sont stockées dans **InfluxDB 3 Core**, une base de données optimisée pour les **séries temporelles**, particulièrement adaptée aux applications IoT. Les mesures sont enregistrées dans la measurement **greenhouse_data** .

Chaque point contient :

- des **tags** (ex : identifiant du capteur, niveau d'alerte)
- des **fields** (température, humidité, niveau d'eau, numéro de séquences

Cette structure permet d'effectuer rapidement des requêtes temporelles et des filtrages par appareil.

Pour des raisons de sécurité, InfluxDB n'est pas exposée sur Internet et reste accessible uniquement depuis le réseau interne des conteneurs.

Visualisation

La visualisation des données est assurée par **Grafana**, configuré automatiquement au démarrage. Un tableau de bord principal affiche les informations issues des capteurs sous forme de graphiques. Les panels principaux permettent de suivre :

- **la température**
- **l'humidité**
- **le niveau d'eau**
- **les alertes détectées**
- **le numéro de séquence des messages**

Ces graphiques permettent de surveiller l'état de la serre et de détecter rapidement des anomalies comme un niveau d'eau trop faible ou une perte de paquets.

Accès réseau

Le service **Nginx** agit comme **reverse proxy** et constitue l'unique point d'entrée du serveur. Il assure la redirection vers les différents services :

Chemin	Service
`/`	page d'accueil
`/api/`	API Django
`/ui/`	Grafana
`/rabbitmq/`	interface RabbitMQ

Toutes les requêtes HTTP sont automatiquement redirigées vers HTTPS afin de garantir une communication chiffrée.

Sécurité

Plusieurs mesures ont été mises en place pour sécuriser l'infrastructure.

Le serveur utilise des certificats **Let's Encrypt** afin de fournir une connexion HTTPS sécurisée. Les protocoles TLS récents sont privilégiés afin d'assurer un niveau de sécurité élevé.

Les informations sensibles (clés API, identifiants, tokens) sont stockées dans un fichier `.env` non versionné afin d'éviter leur présence dans le code source.

De plus, certains services comme **InfluxDB** ne sont pas accessibles directement depuis Internet et restent limités au réseau interne Docker.

Déploiement

L'ensemble de l'infrastructure est défini dans un fichier **docker-compose.yaml** qui orchestre les différents conteneurs :

- nginx
- api
- worker
- rabbitmq
- influxdb
- grafana

Les données persistantes sont stockées dans des **volumes Docker**, ce qui permet de conserver les données même après redéploiement.

Le déploiement est simple et se fait en trois étapes :

1. configuration du fichier `.env`
2. vérification des certificats HTTPS
3. lancement de l'infrastructure :

docker compose up -d